Python

Study Booklet

Redesigned from classroom notes into a

modern study guide

Cleaned, reorganized, repeated content merged, weak pages omitted.

Preface

Welcome to the Python Study Booklet. This guide is a carefully reconstructed and polished version of original classroom notes. The aim is to provide a clean, typed, and structured approach to mastering Python fundamentals from beginner to early-intermediate levels.

Programming often feels overwhelming until you recognize the recurring patterns of logic. Instead of memorizing hundreds of specific scripts, this booklet focuses on the core building blocks: Variables, Control Flow, Functions, Data Structures, and Object-Oriented concepts.

How this booklet is organized:

- **Chapters 1-8:** Core concepts with concise explanations, syntax examples, and typical use cases.
- **Final Revision Toolkit:** A quick-reference syntax sheet, debugging checklist, and pattern map.
- **Practice Problems:** A mixed set of 12 questions to test your logic and syntax recall.
- **Model Solutions:** Concise outlines and code snippets for all practice problems.

Table of Contents

Chapter 1: Python Foundations Page 4

Chapter 2: Variables, Data Types, and Input/Output Page 5

Chapter 3: Conditions and Loops Page 6

Chapter 4: Functions and Problem Solving Page 7

Chapter 5: Strings, Lists, Tuples, Sets, and Dictionaries Page 8

Chapter 6: File Handling and Error Handling Page 9

Chapter 7: Object-Oriented Programming Basics Page 10

Chapter 8: Useful Python Tools Page 11

Final Revision Toolkit Page 12

Practice Problems Page 13

Model Solutions Page 14

Final Advice Page 15

CHAPTER 1

Python Foundations

Python is an interpreted, high-level, general-purpose programming language. Its design philosophy emphasizes code readability with its use of significant indentation. Python is dynamically typed and garbage-collected.

What Python Is (and Isn't)

Python is designed to be highly readable. It abstracts away memory management (like pointers in C/C++) to let the programmer focus on problem-solving logic. However, because it is interpreted, it is generally slower than compiled languages like Java or C++.

REPL vs. Script

You can execute Python code in two main ways:

- **REPL (Read-Eval-Print Loop):** Typing `python` in your terminal opens an interactive shell. It executes statements one by one. Great for testing small snippets.
- **Script:** Writing code in a text file with a `.py` extension and running it via `python my_script.py`. This is how actual programs are built.

Indentation and Comments

Unlike languages that use curly braces `{}` to define blocks of code, Python uses whitespace (indentation). A standard indent is 4 spaces.

```
# This is a single-line comment
if True:
    print("This block is indented") # 4 spaces is standard
    if True:
        print("Nested block requires 8 spaces")
```

> **EXAM LENS**
>
> Indentation errors (`IndentationError`) are the most common beginner mistake. Always ensure consistencyâ€”never mix tabs and spaces in the same file.

Key Takeaways

- Python prioritizes readability and simplicity.
- Code blocks are defined by indentation, not brackets.
- Comments start with `#` and are crucial for documenting your logic.

CHAPTER 2

Variables, Data Types, and I/O

Variables are containers for storing data values. Python has no command for declaring a variable; it is created the moment you first assign a value to it.

Variables and Naming Rules

Variable names must start with a letter or an underscore, and can only contain alpha-numeric characters and underscores. Python convention uses `snake_case` for variable names.

Core Data Types

Type	Description	Example
`int`	Integer (whole numbers)	`x = 5`
`float`	Floating point (decimals)	`y = 3.14`
`str`	String (text)	`name = "Alice"`
`bool`	Boolean (True or False)	`is_valid = True`

Input, Output, and Type Conversion

To display data, use `print()`. To collect data from the user, use `input()`. Note that `input()` **always returns a string**, so you must cast (convert) it if you need a number.

```
age_str = input("Enter your age: ")
age_int = int(age_str)  # Type conversion
print(f"Next year you will be {age_int + 1}") # Using f-string
```

> **QUICK INSIGHT**
>
> **F-strings** (formatted string literals) are the modern way to format output in Python. Just prefix the string with an `f` and put variables directly inside curly braces `{}`.

Key Takeaways

- Variables are created upon assignment. Follow `snake_case` naming.
- Always convert `input()` data to the correct type (like `int()` or `float()`) before doing math.
- Use f-strings for clean, readable output formatting.

CHAPTER 3

Conditions and Loops

Control flow allows your program to make decisions and repeat actions based on dynamic conditions.

If-Elif-Else and Operators

Conditions check boolean expressions using comparison (`==`, `!=`, `>`, `<=`) and logical operators (`and`, `or`, `not`). The first True block executes.

```
score = 85
if score >= 90:
    print("A")
elif score >= 80 and score < 90:
    print("B")
else:
    print("Study harder")
```

For Loops and Range

For Loop: Used for iterating over a sequence (like a list, string, or range). The `range(start, stop, step)` function is heavily used to loop a specific number of times.

```
# Prints 0, 2, 4, 6, 8 (starts at 0, up to but not including 10, step
2)
for i in range(0, 10, 2):
    print(i)
```

While Loops and Control Statements

While Loop: Repeats as long as a condition is True. Useful when the number of iterations is unknown.

```
count = 5
while count > 0:
    if count == 3:
        break       # Exits loop early
    print(count)
    count -= 1
```

> **EXAM LENS**
>
> `break` exits the loop entirely. `continue` skips the rest of the current iteration and jumps to the next one. `pass` does absolutely nothingâ€”it's just a placeholder to prevent syntax errors in empty blocks.

Key Takeaways

- `elif` prevents checking multiple mutually exclusive `if` conditions.
- `range()` is exclusive at the upper bound.
- Beware of infinite `while` loops; ensure your condition will eventually become False.

CHAPTER 4

Functions and Problem Solving

Functions are reusable blocks of code. They help break large, complex problems into smaller, manageable, and testable pieces.

Defining a Function

Use the `def` keyword. Functions can take parameters (inputs) and use `return` to send data back to the caller.

```
def calculate_area(width, height):
    """Calculates the area of a rectangle."""
    area = width * height
    return area

result = calculate_area(5, 10) # 5 and 10 are arguments
print(result) # Output: 50
```

Parameters and Return Types

Parameters act as local variables inside the function. A function ends immediately when it hits a `return` statement. If no return statement is specified, Python implicitly returns `None`.

Scope

Variables created inside a function are *local* to that function. They cannot be accessed from the outside. Variables defined outside all functions are *global*.

> **QUICK INSIGHT**
>
> If you try to modify a global variable inside a function, Python will create a local variable with the same name instead, unless you explicitly use the `global` keyword (which is generally considered bad practice).

Key Takeaways

- Good functions do one thing well. If your function requires the word "and" in its name, split it.
- Use docstrings (`""" """`) right under the definition to explain what the function does.
- Arguments are what you pass in; Parameters are the variables in the function definition.

CHAPTER 5

Strings, Lists, Tuples, Sets, and Dictionaries

Data structures are specialized formats for organizing and storing data. Choosing the right one is 80% of problem-solving.

Strings (Immutable)

Text data. Strings can be sliced using `[start:stop:step]`.

```
text = "Python"
print(text[0:3])    # 'Pyt'
print(text[::-1])   # 'nohtyP' (Reverses string)
```

Lists (Mutable, Ordered)

Arrays of items. Can hold mixed types. Methods include `.append()`, `.pop()`, and `.sort()`.

```
fruits = ["apple", "banana"]
fruits.append("cherry")
fruits[0] = "kiwi" # Lists can be changed
```

Tuples (Immutable, Ordered)

Like lists, but cannot be changed after creation. Uses parentheses `()`. Faster and safer for fixed data (e.g., coordinates).

Sets (Mutable, Unordered, Unique)

Collections of unique items. Great for removing duplicates and fast membership testing (`in` keyword).

```
nums = {1, 2, 2, 3} # Becomes {1, 2, 3}
```

Dictionaries (Key-Value Pairs)

Unordered mappings. Keys must be immutable (like strings/tuples). Extremely fast lookups.

```
student = {"name": "Alice", "age": 20}
student["grade"] = "A" # Adding a new key-value pair
print(student.get("name")) # Safely gets "Alice"
```

> **EXAM LENS**
>
> When tracking frequencies/counts of items, use a Dictionary. When finding unique elements, use a Set. When order matters, use a List.

Key Takeaways

- Mutable means changeable (Lists, Dicts, Sets). Immutable means unchangeable (Strings, Tuples).
- Slicing is a powerful tool for Strings and Lists.

CHAPTER 6

File Handling and Error Handling

Programs must interact with external environments securely and handle unexpected input gracefully.

File Handling with `open()`

The safest way to open a file is using the `with` context manager. It automatically closes the file even if an error occurs.

Common modes: `'r'` (read), `'w'` (write/overwrite), `'a'` (append).

```
# Writing to a file
with open("data.txt", "w") as file:
    file.write("Hello, World!\n")

# Reading from a file
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

Error Handling (Try / Except)

When Python encounters a critical error, it crashes and throws an Exception. We can catch these to keep the program running.

```
try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ValueError:
    print("That was not a valid number!")
except ZeroDivisionError:
    print("You cannot divide by zero!")
finally:
    print("This runs no matter what.")
```

> **QUICK INSIGHT**
>
> Always catch specific exceptions (like `ValueError` or `FileNotFoundError`). Using a bare `except:` hides bugs because it catches everything, including keyboard interrupts.

Key Takeaways

- Always use `with open(...)` to handle files.
- Use `try/except` blocks around code that relies on user input or external resources (like reading files).

CHAPTER 7

Object-Oriented Programming Basics

Object-Oriented Programming (OOP) groups data (attributes) and behavior (methods) into reusable blueprints called Classes.

Classes and Objects

A **Class** is the blueprint. An **Object** (or instance) is the actual item built from that blueprint.

```
class Dog:
    # The __init__ method initializes the object's attributes
    def __init__(self, name, age):
        self.name = name  # Instance attribute
        self.age = age

    # A method (behavior)
    def bark(self):
        return f"{self.name} says Woof!"

# Creating instances (objects)
dog1 = Dog("Buddy", 3)
dog2 = Dog("Lucy", 5)

print(dog1.bark()) # Buddy says Woof!
```

The `self` Parameter

`self` represents the specific object being created or manipulated. It must be the first parameter in any instance method, though you don't pass it in when calling the method (Python handles it automatically).

Why OOP Helps

OOP makes code modular. Through **Inheritance**, a new class can inherit attributes and methods from an existing class, preventing code duplication.

> **EXAM LENS**

> `__init__` is called a constructor. It runs automatically when you instantiate a new object. Remember: attributes are variables belonging to an object; methods are functions belonging to an object.

Key Takeaways

- Classes use `PascalCase` naming (e.g., `BankAccount`).
- `self` is mandatory for instance variables and methods.

CHAPTER 8

Useful Python Tools

Python's true power lies in its syntactic sugar and its massive standard library.

List Comprehensions

A concise way to create lists in a single line. Formula: `[expression for item in iterable if condition]`

```
# Traditional way
squares = []
for i in range(5):
    squares.append(i**2)

# List Comprehension
squares_comp = [i**2 for i in range(5)]
# Evens only: [i for i in range(10) if i % 2 == 0]
```

Enumerate and Zip

`enumerate()` allows you to loop over an iterable and get an automatic counter (index).

```
for index, item in enumerate(["A", "B", "C"]):
    print(index, item)  # Prints 0 A, 1 B, 2 C
```

`zip()` allows you to loop over two or more lists simultaneously.

Importing Modules

Modules are pre-written Python files. The Standard Library comes with Python.

```
import random
import math

print(random.randint(1, 10)) # Random integer between 1 and 10
print(math.sqrt(16))         # 4.0
```

> **QUICK INSIGHT**
>
> Instead of importing an entire module, you can import specific functions to save memory and typing: `from random import choice`.

Key Takeaways

- List comprehensions replace simple for-loops that append to lists.
- Don't reinvent the wheel. Check the Python Standard Library (like `math`, `datetime`, `pathlib`) before writing complex custom logic.

Final Revision Toolkit

Topic Recognition Checklist

- Need to store ordered items? â†’ **List**
- Need unique items only? â†’ **Set**

- Need key-value mapping (e.g., frequencies)? â†' **Dictionary**
- Need to repeat a known number of times? â†' **For Loop**
- Need to repeat until a condition changes? â†' **While Loop**
- Need to read/write external data? â†' **with open()**

Common Syntax Reminders

- `len(x)`: Gets the length of a string, list, dict, etc.
- `type(x)`: Checks the data type of a variable.
- `x.append(y)`: Adds y to the end of list x.
- `range(start, stop, step)`: Ends BEFORE the stop value.

Style & Debugging Checklist

- Are your variable names descriptive? (`student_age` instead of `x`)
- Did you match all your parentheses and quotes?
- Did you accidentally use `=` (assignment) instead of `==` (comparison)?

Beginner Mistakes to Avoid

- **Modifying a list while iterating over it:** Iterate over a copy instead (`for item in my_list[:]:`).
- **Forgetting to cast input:** `input()` is always a string!
- **Inconsistent indentation:** Stick to 4 spaces strictly.

Practice Problems

1. **Variables:** Swap the values of two variables `a = 5` and `b = 10` without hardcoding the numbers.
2. **Conditionals:** Write logic to check if a number `n` is even or odd.
3. **Loops:** Print all numbers from 1 to 10 using a while loop.

4. **Functions:** Write a function `is_prime(n)` that returns True if a number is prime, False otherwise.
5. **Strings:** Write a function that checks if a string is a palindrome (reads the same forwards and backwards).
6. **Lists:** Given a list of numbers, return a new list containing only the positive numbers.
7. **Dictionaries:** Write a function that counts the frequency of each character in a string and returns a dictionary.
8. **Sets:** Find the common elements between two lists, returning them as a list.
9. **File I/O:** Write code to append the string "End of log" to a file called `log.txt`.
10. **Exceptions:** Write a safe integer division function that catches division by zero and non-integer inputs.
11. **OOP:** Create a `Car` class with a `make`, `model`, and a method `drive()` that prints "Vroom!".
12. **Comprehensions:** Use a list comprehension to square all even numbers from 0 to 10.

Model Solutions

1. Variables:

`a, b = b, a`

2. Conditionals:

`if n % 2 == 0: print("Even") else: print("Odd")`

3. Loops:

```
i = 1
while i <= 10:
    print(i)
    i += 1
```

4. Functions:

```
def is_prime(n):
    if n < 2: return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0: return False
    return True
```

5. Strings:

```
def is_palindrome(s): return s == s[::-1]
```

6. Lists:

```
def get_positives(nums): return [n for n in nums if n > 0]
```

7. Dictionaries:

```
def char_frequency(s):
    freq = {}
    for char in s:
        freq[char] = freq.get(char, 0) + 1
    return freq
```

8. Sets:

```
def common_elements(list1, list2): return list(set(list1) & set(list2))
```

9. File I/O:

```
with open("log.txt", "a") as f:
    f.write("End of log\n")
```

10. Exceptions:

```
def safe_divide(a, b):
    try:
        return int(a) / int(b)
    except ZeroDivisionError:
        return "Cannot divide by zero"
    except ValueError:
        return "Inputs must be numbers"
```

11. OOP:

```
class Car:
    def __init__(self, make, model):
        self.make = make
        self.model = model
    def drive(self):
        print("Vroom!")
```

12. Comprehensions:

```
squares = [x**2 for x in range(11) if x % 2 == 0]
```

Final Advice

Programming is learned by doing. Reading this booklet is a solid start, but typing the code out, making mistakes, and fixing errors is where true learning happens. Remember that the computer only does exactly what you tell it to do.

Focus on understanding the error messagesâ€”they are clues, not obstacles. Break large problems into small, testable chunks. Take breaks when you are stuck. **Good luck with your Python journey!**